E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

1 Code Debugging

Part A – Identifying the Issues

After going through the code, I found a bunch of problems — some are straightforward bugs, some are more subtle logic issues.

1. No input validation / error handling The code does `data['name']`, `data['sku']`, etc. directly. If any of these keys are missing in the request body, it'll throw a raw `KeyError` and crash with a 500. Same thing if `request.json` is `None` (e.g., wrong Content-Type header).

2. No SKU uniqueness check The context says SKUs must be unique across the platform, but there's zero enforcement in the application layer. If the DB doesn't have a unique constraint either, duplicate SKUs will silently get inserted.

3. Two separate commits = partial failure risk The product gets committed first, then inventory. If anything goes wrong between the two commits (server crash, DB timeout, exception), you end up with a product in the DB that has no inventory record. That's a data integrity nightmare.

4. Price has no type validation Since price can be decimal values, accepting it raw from JSON is risky — someone could pass a string like `"free"` and it'll blow up at the DB layer with a confusing error.

5. `initial_quantity` might be missing The context says some fields are optional, but `data['initial_quantity']` will KeyError if not provided. Should probably default to 0.

6. No HTTP status codes on the response Flask's default return is 200, but a successful resource creation should return **201 Created**. Small thing, but it matters for REST standards and for frontend/API consumers.

7. No warehouse existence validation `warehouse_id` is being trusted blindly. If someone passes a warehouse_id that doesn't exist, you'll get a DB foreign key error (if constraints exist) or silently bad data (if they don't).

8. The response doesn't return enough info Only `product_id` is returned. Given that products can exist in multiple warehouses, it would make more sense to also return at minimum the warehouse and SKU so the caller can confirm what got created.

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

Part B – Production Impact

Issue	What goes wrong
No input validation	API returns 500 instead of a clean 400; logs get polluted; hard to debug
No SKU uniqueness check	Duplicate SKUs corrupt inventory tracking and reporting
Two commits	Orphaned products with no inventory — stock management breaks silently
Price not validated	DB-level errors leak into responses, confusing API consumers
Missing `initial_quantity`	Crashes on valid use cases where quantity isn't supplied upfront
Wrong status code	Frontend/clients that check status codes behave incorrectly
No warehouse check	Foreign key violations or ghost data depending on DB config

Part C – Fixed Version

```
from flask import request, jsonify
from sqlalchemy.exc import IntegrityError
from decimal import Decimal, InvalidOperation

@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.get_json()

    # Basic check — make sure we even got JSON
    if not data:
        return jsonify({"error": "Request body must be valid JSON"}), 400

    # Check required fields
    required_fields = ['name', 'sku', 'price', 'warehouse_id']
    missing = [f for f in required_fields if f not in data]
    if missing:
        return jsonify({"error": f"Missing required fields: {missing}"}), 400
```

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

```
    # Validate price is actually a number
    try:
        price = Decimal(str(data['price']))
        if price < 0:
            return jsonify({"error": "Price cannot be negative"}), 400
    except (InvalidOperation, TypeError):
        return jsonify({"error": "Price must be a valid number"}), 400

    # Check warehouse actually exists
    warehouse = Warehouse.query.get(data['warehouse_id'])
    if not warehouse:
        return jsonify({"error": "Warehouse not found"}), 404

    # Check SKU uniqueness before trying to insert
    existing = Product.query.filter_by(sku=data['sku']).first()
    if existing:
        return jsonify({"error": f"SKU '{data['sku']}' already exists"}), 409

    # Default initial_quantity to 0 if not provided
    initial_quantity = data.get('initial_quantity', 0)

    try:
        # Wrap both inserts in a single transaction
        product = Product(
            name=data['name'],
            sku=data['sku'],
            price=price,
            warehouse_id=data['warehouse_id']
        )
        db.session.add(product)
        db.session.flush()  # gets product.id without committing yet

        inventory = Inventory(
            product_id=product.id,
            warehouse_id=data['warehouse_id'],
            quantity=initial_quantity
        )
        db.session.add(inventory)

        db.session.commit()  # single commit — both succeed or neither does

    except IntegrityError:
        db.session.rollback()
```

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

```
        # Catches any race condition on SKU uniqueness at DB level
        return jsonify({"error": "Could not create product, possible duplicate SKU"}), 409
    except Exception as e:
        db.session.rollback()
        # Don't leak internal error details to the client
        app.logger.error(f"Product creation failed: {str(e)}")
        return jsonify({"error": "Internal server error"}), 500

    return jsonify({
        "message": "Product created successfully",
        "product_id": product.id,
        "sku": product.sku,
        "warehouse_id": data['warehouse_id']
    }), 201
```

Feature / Change	Reasoning
`db.session.flush()`	Allows getting the product ID for the inventory record within a single atomic transaction.
Try/Except Wrapping	Ensures clean rollbacks on failure, preventing orphaned records.
`data.get('quantity', 0)`	Gracefully handles optional fields instead of crashing.
Decimal for price	Avoids floating point inaccuracies for currency values.
Explicit 409 Status	Properly handles duplicate SKU conflicts at both app and DB levels.
201 Status Code	Adheres to REST standards for successful resource creation.

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

2 Database Design

Schema (SQL DDL)

I went with SQL DDL.

```
-- Core company struct.
CREATE TABLE companies (
    id            SERIAL PRIMARY KEY,
    name          VARCHAR(255) NOT NULL,
    email         VARCHAR(255) UNIQUE,
    phone         VARCHAR(50),
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE warehouses (
    id            SERIAL PRIMARY KEY,
    company_id    INT NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
    name          VARCHAR(255) NOT NULL,
    address_line1 VARCHAR(255),
    address_line2 VARCHAR(255),
    city          VARCHAR(100),
    country       VARCHAR(100),
    postal_code   VARCHAR(20),
    is_active     BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Suppliers are separate from companies (a supplier might supply multiple companies)
CREATE TABLE suppliers (
    id            SERIAL PRIMARY KEY,
    name          VARCHAR(255) NOT NULL,
    contact_email VARCHAR(255),
    contact_phone VARCHAR(50),
    address       TEXT,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Links which suppliers supply which companies (many-to-many)
CREATE TABLE company_suppliers (
    company_id    INT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
    supplier_id   INT NOT NULL REFERENCES suppliers(id) ON DELETE CASCADE,
    since         DATE,
    PRIMARY KEY (company_id, supplier_id)
```

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

```
);

-- Products catalog (what exists, not where or how many)
CREATE TABLE products (
    id            SERIAL PRIMARY KEY,
    sku           VARCHAR(100) UNIQUE NOT NULL,
    name          VARCHAR(255) NOT NULL,
    description   TEXT,
    price         NUMERIC(12, 2) NOT NULL CHECK (price >= 0),
    is_bundle     BOOLEAN DEFAULT FALSE,
    supplier_id   INT REFERENCES suppliers(id) ON DELETE SET NULL,
    is_active     BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Bundle contents: which products make up a bundle
-- A bundle can contain other bundles (recursive), handled at app layer
CREATE TABLE bundle_items (
    bundle_id     INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    component_id  INT NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
    quantity      INT NOT NULL DEFAULT 1 CHECK (quantity > 0),
    PRIMARY KEY (bundle_id, component_id),
    CHECK (bundle_id != component_id)   -- a product can't contain itself
);

-- Inventory: how much of a product is at each warehouse
CREATE TABLE inventory (
    id            SERIAL PRIMARY KEY,
    product_id    INT NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
    warehouse_id  INT NOT NULL REFERENCES warehouses(id) ON DELETE RESTRICT,
    quantity      INT NOT NULL DEFAULT 0 CHECK (quantity >= 0),
    reorder_level INT DEFAULT 0,      -- optional low-stock threshold
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (product_id, warehouse_id)   -- one record per product per warehouse
);

-- Audit log: every time inventory changes, we write a row here
CREATE TABLE inventory_logs (
    id            SERIAL PRIMARY KEY,
    inventory_id  INT NOT NULL REFERENCES inventory(id) ON DELETE RESTRICT,
    product_id    INT NOT NULL,
    warehouse_id  INT NOT NULL,
    change_type   VARCHAR(50) NOT NULL,   -- 'restock', 'sale', 'transfer', 'adjustment', etc.
```

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

```
    quantity_before INT NOT NULL,
    quantity_change INT NOT NULL,          -- positive = added, negative = removed
    quantity_after  INT NOT NULL,
    reason          TEXT,
    changed_by      INT,                   -- user_id if you have auth, NULL for now
    changed_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indexes
CREATE INDEX idx_inventory_product     ON inventory(product_id);
CREATE INDEX idx_inventory_warehouse   ON inventory(warehouse_id);
CREATE INDEX idx_inventory_logs_inv    ON inventory_logs(inventory_id);
CREATE INDEX idx_inventory_logs_time   ON inventory_logs(changed_at);
CREATE INDEX idx_products_sku          ON products(sku);
CREATE INDEX idx_products_supplier     ON products(supplier_id);
CREATE INDEX idx_warehouses_company    ON warehouses(company_id);
```

ERD (Text Representation)

companies ──< warehouses
companies ──< company_suppliers >── suppliers
suppliers ──< products
products ──< inventory >── warehouses
products ──< bundle_items >── products (self-referencing)
inventory ──< inventory_logs

Gaps I'd Raise With the Product Team

These are the things I noticed were either missing or ambiguous in the requirements — I'd need answers before finalizing the schema:

1. What exactly is the relationship between suppliers and companies? Right now I assumed a supplier can work with multiple companies and a company can have multiple suppliers (many-to-many). But if a supplier is always exclusive to one company, a simple `company_id` FK on `suppliers` is cleaner. Which is it?

2. Does a product belong to one supplier, or can multiple suppliers provide the same product? I've currently got one `supplier_id` on `products`. But in practice, the same SKU (e.g., a generic cable) might be sourced from different suppliers at different times or for different warehouses. If that's the case, we need a `product_suppliers` junction table instead.

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

3. What triggers an inventory log entry — application code or a DB trigger? I'd recommend a DB trigger on `inventory.quantity` so nothing slips through, but that has tradeoffs with visibility and debugging. Worth aligning on early.

4. Can bundles contain other bundles (nested bundles)? My schema technically allows it since `bundle_items` just references `products` and any product can be a bundle. But if nested bundles are valid, the app needs to handle infinite loop prevention (A contains B, B contains A). Is that a real use case?

5. What does "track when inventory changes" actually mean for the business? Is this just for debugging/audit purposes, or does it power something like reports, notifications (low stock alerts), or customer-facing stock history? That affects how much detail the log needs and how long we retain it.

6. Are there user accounts / authentication? The `changed_by` column in `inventory_logs` is currently just an int placeholder. If there's a users table eventually, this should be a proper FK. If it's a public-facing API with no auth yet, we might just log API keys or skip it for now.

7. What happens to inventory when a warehouse is deactivated? I used `is_active` on warehouses instead of hard deletes, but what should happen to existing inventory records? Zero them out? Archive them? Transfer to another warehouse?

8. Is pricing per product global, or can it vary by warehouse/company? Right now price lives on the `products` table (one global price). But some businesses price differently by region or warehouse. Is that needed here?

Design Decisions Explained

`NUMERIC(12,2)` for price, not FLOAT Floating point types can't represent values like `9.99` exactly in binary, which leads to subtle rounding bugs in financial calculations. `NUMERIC` is exact — always use it for money.

Separate `inventory` and `inventory_logs` tables `inventory` is the current state (fast reads, single row per product/warehouse), while `inventory_logs` is the full history (append-only, never updated). Mixing them would make both slower and harder to query.

`UNIQUE(product_id, warehouse_id)` on **inventory** Enforces at the DB level that there's only one inventory record per product per warehouse. Without this, application bugs can silently create duplicates and give you wrong stock counts.

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

`ON DELETE RESTRICT` on inventory and logs I don't want a product or warehouse getting deleted if inventory records reference it. That data is historically important — the right flow is to deactivate (`is_active = FALSE`) rather than hard delete.

Self-referencing `bundle_items` table Cleaner than adding bundle logic into the products table itself. Keeping bundles as a separate relationship means a regular product can become a bundle without schema changes, and querying bundle contents is straightforward.

Indexes on foreign keys and timestamp columns Any column you're frequently joining on or filtering by should be indexed. `changed_at` on logs especially — you'll almost always be querying logs within a time range.

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

3 API Implimentation

Assumptions I'm Making (since requirements were incomplete)

Before jumping into code, here's what I had to assume — I'd validate all of these with the team:

- **"Recent sales activity"** = product had at least one sale in the last **30 days**. This is a guess — it could be 7, 14, 60 days. I've made it a configurable constant so it's easy to change.
- **Low stock threshold** varies by product type — I'm assuming there's a **`reorder_level`** column on the `inventory` table (which I designed in Part 2). If the product is a bundle, threshold is higher (I'm using a **`bundle_threshold_multiplier`**).
- **Days until stockout** = **`current_stock / avg_daily_sales`** over the last 30 days. If there are zero recent sales, I skip the alert entirely (per the "recent sales activity" rule).
- **`inventory_logs`** is the source of truth for sales — specifically rows where **`change_type = 'sale'`.**
- The `company_id` in the URL refers to the company that *owns* the warehouses, not the supplier.
- A product alert is **per warehouse**, not aggregated across all warehouses. A product might be fine in one warehouse but critically low in another.

Implementation

```
from flask import Blueprint, jsonify, g
from sqlalchemy import text
from datetime import datetime, timedelta
from functools import wraps

alerts_bp = Blueprint('alerts', __name__)

# --- Config constants (easy to change without touching logic) ---
RECENT_SALES_WINDOW_DAYS = 30
BUNDLE_THRESHOLD_MULTIPLIER = 1.5   # bundles need higher buffer since they depend on components
DEFAULT_REORDER_LEVEL = 10          # fallback if reorder_level isn't set on inventory row
MAX_ALERTS_PER_RESPONSE = 500       # safety cap — don't return 10k rows accidentally


def require_company_access(f):
```

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

```
    """
    Placeholder auth decorator.
    In a real app this would verify the requesting user
    actually belongs to this company. Skipping full auth
    implementation here but flagging it as necessary.
    """
    @wraps(f)
    def decorated(*args, **kwargs):
        # TODO: check g.current_user.company_id == company_id
        return f(*args, **kwargs)
    return decorated


@alerts_bp.route('/api/companies/<int:company_id>/alerts/low-stock', methods=['GET'])
@require_company_access
def get_low_stock_alerts(company_id):
    """
    Returns low-stock alerts for all warehouses belonging to a company.
    Only includes products with recent sales activity (last 30 days).
    Calculates days until stockout based on avg daily sales velocity.
    """

    # First, make sure the company actually exists
    # Avoids leaking info — return 404 rather than an empty alerts list
    company = Company.query.get(company_id)
    if not company:
        return jsonify({"error": "Company not found"}), 404

    sales_window_start = datetime.utcnow() -
timedelta(days=RECENT_SALES_WINDOW_DAYS)

    try:
        # Single query to pull everything we need.
        # I'm using raw SQL here because this is a read-heavy analytics query
        # with multiple joins and aggregations — SQLAlchemy ORM would make this
        # significantly harder to read and optimize.
        query = text("""
            WITH recent_sales AS (
                -- Aggregate sales per product per warehouse over the window
                -- quantity_change is negative for sales, so we flip it with ABS()
                SELECT
                    il.product_id,
                    il.warehouse_id,
                    SUM(ABS(il.quantity_change))                    AS total_sold,
```

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

```
        COUNT(DISTINCT DATE(il.changed_at))              AS active_days,
        -- Daily velocity: total sold / days in window (not just active days)
        -- Using full window length avoids inflating velocity for bursty products
        ROUND(
          SUM(ABS(il.quantity_change))::NUMERIC / :window_days,
          2
        )                                        AS avg_daily_sales
      FROM inventory_logs il
      WHERE
        il.change_type = 'sale'
        AND il.changed_at >= :sales_window_start
      GROUP BY il.product_id, il.warehouse_id
    )

    SELECT
      p.id                        AS product_id,
      p.name                      AS product_name,
      p.sku,
      p.is_bundle,
      w.id                        AS warehouse_id,
      w.name                      AS warehouse_name,
      inv.quantity                AS current_stock,
      inv.reorder_level,
      rs.avg_daily_sales,
      rs.total_sold,
      s.id                        AS supplier_id,
      s.name                      AS supplier_name,
      s.contact_email             AS supplier_email

    FROM inventory inv

    -- Only warehouses that belong to this company
    JOIN warehouses w
      ON inv.warehouse_id = w.id
      AND w.company_id = :company_id
      AND w.is_active = TRUE

    JOIN products p
      ON inv.product_id = p.id
      AND p.is_active = TRUE

    -- Inner join: this filters out products with NO recent sales (business rule)
    JOIN recent_sales rs
      ON rs.product_id = inv.product_id
```

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

```
            AND rs.warehouse_id = inv.warehouse_id

        -- Supplier info for reordering — LEFT JOIN because supplier might not be set
        LEFT JOIN suppliers s
            ON p.supplier_id = s.id

        WHERE
            -- Only return rows where stock is actually below threshold
            -- Threshold check happens here to avoid fetching everything into Python
            inv.quantity <= CASE
                WHEN p.is_bundle = TRUE
                    THEN COALESCE(inv.reorder_level, :default_reorder) * :bundle_multiplier
                ELSE
                    COALESCE(inv.reorder_level, :default_reorder)
            END

        ORDER BY
            -- Most urgent first: lowest stock relative to threshold
            (inv.quantity::FLOAT / NULLIF(
                CASE
                    WHEN p.is_bundle THEN COALESCE(inv.reorder_level, :default_reorder) *
:bundle_multiplier
                    ELSE COALESCE(inv.reorder_level, :default_reorder)
                END,
            0)) ASC

        LIMIT :max_alerts
    """)

    results = db.session.execute(query, {
        "company_id":          company_id,
        "sales_window_start":   sales_window_start,
        "window_days":          RECENT_SALES_WINDOW_DAYS,
        "default_reorder":      DEFAULT_REORDER_LEVEL,
        "bundle_multiplier":    BUNDLE_THRESHOLD_MULTIPLIER,
        "max_alerts":           MAX_ALERTS_PER_RESPONSE,
    }).fetchall()

  except Exception as e:
    # Log the real error internally, return a clean message to the client
    app.logger.error(f"Low stock query failed for company {company_id}: {str(e)}")
    return jsonify({"error": "Failed to fetch alerts"}), 500

  alerts = []
```

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

```
    for row in results:
        # Calculate days until stockout
        # If avg_daily_sales is somehow 0 (edge case — shouldn't happen due to
        # the JOIN filter but being safe), we skip or set to None
        if row.avg_daily_sales and row.avg_daily_sales > 0:
            days_until_stockout = round(row.current_stock / row.avg_daily_sales)
        else:
            days_until_stockout = None

        # Recalculate threshold cleanly in Python for the response field
        base_threshold = row.reorder_level if row.reorder_level else
DEFAULT_REORDER_LEVEL
        threshold = int(base_threshold * BUNDLE_THRESHOLD_MULTIPLIER) if row.is_bundle
else base_threshold

        alerts.append({
            "product_id":          row.product_id,
            "product_name":        row.product_name,
            "sku":                 row.sku,
            "warehouse_id":        row.warehouse_id,
            "warehouse_name":      row.warehouse_name,
            "current_stock":       row.current_stock,
            "threshold":           threshold,
            "days_until_stockout":  days_until_stockout,
            "supplier": {
                "id":              row.supplier_id,
                "name":            row.supplier_name,
                "contact_email":   row.supplier_email,
            } if row.supplier_id else None   # don't return empty supplier object
        })

    return jsonify({
        "alerts":       alerts,
        "total_alerts": len(alerts),
    }), 200
```

Edge Cases Handled

Company doesn't exist Returns a clean 404 instead of an empty `alerts: []`, which would be misleading — there's a difference between "no alerts" and "company not found."

E-mail: dwivedipavitra92@gmail.com
Link for code: https://github.com/PavitraDwivedi2005/bynryAssesment

Product has no supplier set Returns `"supplier": null` instead of crashing or returning an empty object. The caller needs to handle this — they should show something like "No supplier on record" in the UI.

`avg_daily_sales` is 0 or null Guarded against divide-by-zero. Sets **`days_until_stockout` to `null`** rather than crashing. This shouldn't happen because of the **`JOIN recent_sales`** filter, but defensive coding is still worth it.

Inactive warehouses / products Filtered out at the query level with `is_active = TRUE` — no need to check in Python.

No alerts exist Return**s `{"alerts": [], "total_alerts": 0}`** with a 200. This is correct — absence of alerts is a valid successful response, not an error.

DB failure Wrapped in try/except, logs the real error, returns a clean 500 without leaking stack traces.

Result set too large `LIMIT 500` as a safety cap. In a real system I'd add proper pagination **(`?page=1&page_size=50`)** but flagged it rather than over-engineering for the assessment.

What I'd Add If This Were Production

A few things I left out intentionally to keep this scoped, but worth flagging:

- **Pagination** — **`?page`** and **`?page_size` params, return `total_count`** separately from the capped result
- **Caching** — this query could be expensive at scale. A 5-minute Redis cache per **`company_id`** would help a lot
- **Auth middleware** — the decorator is a placeholder; real implementation needs to verify the user belongs to the company
- **Alert severity levels** — could add **`"severity": "critical"` vs `"warning"`** based on how far below threshold the stock is, or how few days until stockout
- **Unit tests** — specifically for the stockout calculation, the bundle multiplier logic, and the "no recent sales" filter behavior